

Overview

Agentic AI refers to AI systems that do not only generate text but also pursue goals over multiple steps by selecting actions, using tools, and adapting based on observations. A practical definition used across recent courses and industry docs is: an agentic system receives a goal, decomposes it into subproblems, executes actions in an environment, and updates behavior from feedback.

A clean formalization is:

$$\mathcal{A} = (\mathcal{O}, \mathcal{S}, \mathcal{T}, \mathcal{P}, \mathcal{U})$$

Where:

- $\mathcal{O}$ is an observation space (what the agent can perceive).
- $\mathcal{S}$ is an internal state representation.
- $\mathcal{T}$ is a tool/action set (APIs, search, database, code execution).
- $\mathcal{P}$ is a policy or decision function.
- $\mathcal{U}$ is an objective (reward, utility, or task success metric).

Agentic AI vs LLM Chat vs Traditional ML

- Traditional ML predicts one output per input (for example fraud score).
- A chatbot usually maps prompt -> response.
- Agentic AI runs a control loop: goal -> plan -> act -> observe -> revise.

Relationship to prior lessons:

- LLMs provide reasoning and language interfaces.
- RAG provides retrieval-grounded context.
- Classical ML provides scoring, ranking, and anomaly detectors that agents can call as tools.

Agent Foundations

Classical Agent Model

Classical AI defines an agent as something that perceives and acts in an environment to maximize expected utility.

- Environment: external world, APIs, files, humans.
- Perception: sensor input, messages, retrieved docs.
- Internal state: memory of progress and constraints.
- Policy: mapping from state to action.
- Action: tool call, message, decision.

Think of the architecture like a self-driving workflow engine: sensors (inputs), planner (policy), actuators (tools), and telemetry (feedback).

Types of Agents

1. Reactive agents
 - Rule-based or stimulus-response.
 - Fast but brittle for long-horizon tasks.
2. Deliberative agents
 - Maintain explicit world model and plans.
 - Better at multi-step dependencies.
3. Hybrid agents
 - Mix reactive guardrails with deliberate planning.

- Common in enterprise systems.

Single-Agent vs Multi-Agent

- Single-agent systems are easier to reason about and operate.
- Multi-agent systems split responsibilities (planner, researcher, verifier, executor) and can improve modularity, but raise coordination overhead.

A useful rule: start single-agent first; add more agents only when specialization clearly improves reliability or latency.

LLM-Powered Agents

LLMs become agents when wrapped in operational scaffolding:

1. Goal framing: define objective and constraints.
2. Planning: create ordered tasks or dynamic decision points.
3. Tool use: execute actions beyond text generation.
4. Memory: retain short-term and long-term context.
5. Reflection and correction: detect errors and retry.

Chatbot vs Agent

A chatbot answers. An agent owns outcome completion.

Example:

- Chatbot: “Here is a refund policy summary.”
- Agent: “I checked order status, validated eligibility, drafted refund request, and queued escalation because amount exceeded threshold.”

Core Architectural Patterns

Pattern 1: Plan-Act-Observe Loop

Text diagram:

1. Input goal enters planner.
2. Planner chooses next action.
3. Tool executes action.
4. Observation updates state.
5. Loop continues until stop criteria.

Pseudo-math:

$$s_{t+1} = f(s_t, a_t, o_{t+1}), \quad a_t = \pi(s_t)$$

Pattern 2: Planner-Executor Separation

- Planner agent builds task graph.
- Executor agent runs tool calls.
- Critic/verifier checks outputs.

This separation improves debuggability and lets you swap models per role (cheap model for routing, stronger model for synthesis).

Pattern 3: Tool-Augmented Agent

Agent selects from tools such as:

- search
- SQL query
- calendar API
- CRM update
- calculator

Tool interface quality strongly affects reliability. Use typed schemas and explicit error returns.

Pattern 4: Guarded Autonomy

- Hard constraints: banned tools, budget caps, rate limits.
- Soft constraints: confidence thresholds, human approval checkpoints.

Capabilities & Limitations

Where Agentic AI Performs Well

- Workflow automation with multiple systems (ticketing + CRM + email).
- Research workflows (retrieve, summarize, compare, cite).
- Operations triage (classify issue, gather evidence, suggest action).

Reliability Constraints

- Long chains compound small errors.
- Tool misuse causes expensive failures.
- Hidden state bugs reduce reproducibility.

Safety and Governance Constraints

- Prompt injection and tool abuse risks.
- Access control violations.
- Weak auditability if traces are not captured.

Operational principle: autonomy should scale with confidence and observability, not with hype.

Design Framework for Applied Teams

Use this 6-step framework:

1. Define objective and failure cost.
2. Decompose task into deterministic and uncertain steps.
3. Assign tool permissions and guardrails.
4. Add memory boundaries and retention policy.
5. Instrument logs, traces, and evaluation tasks.
6. Introduce human-in-the-loop for high-risk actions.

This framework keeps architecture aligned with business risk.

Business Case Studies & Exceptions

Case Study 1: Email Triage and Scheduling Agent

Scenario:

- A consulting team receives 600+ inbound emails weekly.

Agent design:

- Classify intent (meeting, request, escalation).
- Retrieve prior context from CRM.
- Draft responses and propose calendar slots.
- Escalate ambiguous cases to humans.

Impact:

- Reduced time spent on repetitive coordination.
- Faster first response time.

Exceptions:

- Legal or contractual language requires human approval.
- VIP or sensitive accounts bypass full autonomy.

Case Study 2: Enterprise Onboarding Orchestration

Scenario:

- New customer onboarding requires KYC checks, contract setup, and product provisioning.

Agent system:

- Planner agent decomposes onboarding stages.
- Tool agents call internal APIs.
- Compliance verifier checks required artifacts.

Benefits:

- Shorter onboarding cycle.
- Better traceability for audits.

Risks:

- Incorrect API calls can create inconsistent records.
- Missing human checkpoints can violate compliance.

Interview Questions & Answers

1. Define agentic AI in your own words.

Agentic AI is an AI system that actively pursues goals over multiple steps by planning actions, using tools, and adapting to feedback instead of producing one-off responses.

2. What are the core components of an AI agent architecture?

Environment interface, observation pipeline, internal state/memory, decision policy, action/tool layer, and objective/termination criteria.

3. How is a chatbot different from an agent?

A chatbot focuses on response generation; an agent focuses on outcome completion through iterative action.

4. **What is a plan-act-observe loop?**
A control loop where the agent plans next action, executes it, observes result, updates state, and repeats until goal completion.
5. **Why is tool calling central to agentic systems?**
Because real tasks require external actions such as database reads, API calls, and transactional updates.
6. **When should you avoid multi-agent architecture?**
When one agent can handle the task reliably; multi-agent systems add complexity and coordination overhead.
7. **What is guarded autonomy?**
Allowing autonomous execution under explicit safety constraints like approval gates, policy checks, and budget limits.
8. **Name a common reliability failure in agents.**
Error accumulation across long tool chains where each small mistake propagates downstream.
9. **How do you evaluate an agent?**
Use task success rate, step efficiency, cost per task, latency, policy violations, and human override frequency.
10. **Why does observability matter for agentic AI?**
Without traces and logs, teams cannot debug action choices, reproduce failures, or pass compliance audits.
11. **What is the role of memory in an agent?**
Memory keeps contextual facts and intermediate decisions so the agent can remain coherent over multi-step tasks.
12. **What is a hybrid agent?**
An architecture combining reactive rules for safety/latency and deliberative planning for complex decisions.
13. **How would you reduce hallucinations in agent workflows?**
Use retrieval grounding, tool-verified data, schema-constrained outputs, and verifier steps before final action.
14. **What organizational roles are involved in production agentic AI?**
Product, AI/ML engineering, platform/MLOps, security, and domain operations teams.
15. **How do you decide autonomy level?**
Base it on failure cost, confidence thresholds, regulatory constraints, and quality of monitoring.

References

- NYU Stern Foundations of AI Agents course page: <https://aiagents.stern.nyu.edu/>
- NYU Stern 2026 syllabus PDF: https://pages.stern.nyu.edu/~ilobel/Foundations_of_AI_Agents.pdf
- JHU Imagine Agentic AI course page: <https://imagine.jhu.edu/classes/agentic-ai-fundamentals-architectures-frameworks-and-applications/>
- Harvard Online Agentic AI Foundations: <https://harvardonline.harvard.edu/course/agentic-ai-foundations>
- DeepLearning.AI Agentic AI course: <https://www.deeplearning.ai/courses/agentic-ai>
- LangGraph docs (graph API + workflows): <https://docs.langchain.com/oss/python/langgraph/graph-api> and <https://docs.langchain.com/oss/python/langgraph/workflows-agents>
- CrewAI docs: <https://docs.crewai.com/>
- AutoGen core user guide: <https://microsoft.github.io/autogen/stable/user-guide/core-user-guide/index.html>
- Anthropic: Building Effective AI Agents: <https://resources.anthropic.com/building-effective-ai-agents>

- Anthropic: Effective Context Engineering: <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>